

ASHLEY DAYANNE RODRIGUEZ ACOSTA

NIMSI MAHYLI CONTRERAS MARTÍNEZ

∟ REPORTE DEL
**PROYECTO
FINAL**

+

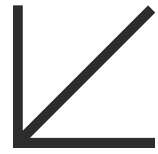
16 de Enero de 2026

ÍNDICE

INTRODUCCIÓN	01
OBJETIVO DEL PROYECTO	02
TECNOLOGÍA UTILIZADA	03
ALGORITMOS Y ESTRUCTURAS IMPLEMENTADAS	04
METODOLOGÍA DE MEDICIÓN	05
RESULTADOS OBTENIDOS	06
PSEUDOCÓDIGOS	09
CONCLUSIÓN	12

01

INTRODUCCIÓN

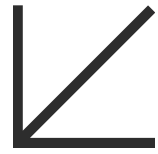


El presente reporte documenta el desarrollo y análisis de una aplicación en Python cuyo objetivo es medir, comparar y validar empíricamente la eficiencia temporal de algoritmos y estructuras de datos fundamentales. El estudio se realiza conforme a los lineamientos establecidos en el documento oficial del Proyecto Final de Estructura de Datos.

La aplicación implementa algoritmos de ordenamiento, búsqueda y estructuras lineales, utilizando mediciones de alta precisión para contrastar los resultados empíricos con las complejidades teóricas conocidas.



02



OBJETIVO DEL PROYECTO

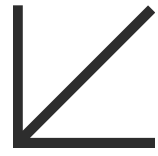
Construir una herramienta que:

- Mida tiempos de ejecución usando temporización de alta precisión.
- Compare algoritmos de diferente complejidad temporal.
- Visualice los resultados mediante tablas y gráficas.
- Valide empíricamente las complejidades $O(1)$, $O(n)$, $O(n \log n)$ y $O(n^2)$.



03

TECNOLOGÍA UTILIZADA



Lenguaje: Python 3.12

Librerías:

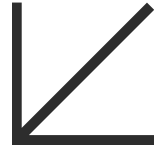
time (medición de tiempo con perf_counter)

random (generación de datos de prueba)

matplotlib (visualización gráfica)



04



ALGORITMOS Y ESTRUCTURAS IMPLEMENTADAS

4.1 Algoritmos de Ordenamiento

- Insertion Sort – $O(n^2)$: Algoritmo de ordenamiento cuadrático, altamente sensible al tamaño de los datos y al peor caso.
- Merge Sort – $O(n \log n)$: Algoritmo eficiente basado en la técnica divide y vencerás, estable incluso en el peor caso.

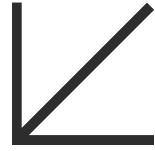
4.2 Algoritmos de Búsqueda

- Búsqueda Secuencial – $O(n)$
- Búsqueda Binaria – $O(\log n)$ (requiere arreglo ordenado)

4.3 Estructuras Lineales

- Pila (Stack): operaciones Push y Pop.
- Cola (Queue): operaciones Enqueue y Dequeue.

05



METODOLOGÍA DE MEDICIÓN

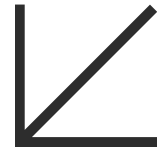
Se implementó una función de medición basada en `time.perf_counter()` para obtener tiempos de alta precisión. Cada algoritmo fue ejecutado múltiples veces y se utilizó el tiempo promedio para reducir variaciones del sistema.

Se consideraron los siguientes escenarios:

- Mejor Caso (MC)
- Caso Promedio (CP)
- Peor Caso (PC)



06



RESULTADOS OBTENIDOS

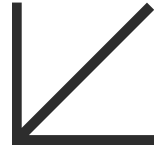
6.1 Estructuras Lineales ($O(1)$) siendo $N = 20,000$

Operación	Tiempo (s)
Push	~0.008 s
Pop	~0.007 s
Enqueue	~0.006 s
Dequeue	~1.40 s

El costo unitario promedio confirma empíricamente la complejidad $O(1)$.



07



RESULTADOS OBTENIDOS

6.2 Algoritmos de Búsqueda donde $N = 100,000$ (Peor Caso)

Algoritmo	Tiempo (s)
Búsqueda Secuencial $O(n)$	~ 0.024 s
Búsqueda Binaria $O(\log n)$	~ 0.000011 s

La búsqueda binaria resultó más de 2000 veces más rápida, validando claramente la ventaja de la complejidad logarítmica.

6.3 Comparación de Escenarios donde $N = 10,000$

Insertion Sort $O(n^2)$:

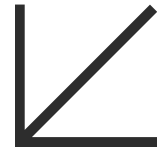
- Mejor Caso: bajo
- Caso Promedio: alto
- Peor Caso: extremadamente alto

Nota: $N=10^6$ es muy lento para este tipo de algoritmo en Python. Por lo tanto se utilizaron valores crecientes significativos sin llegar a este caso

Merge Sort $O(n \log n)$:

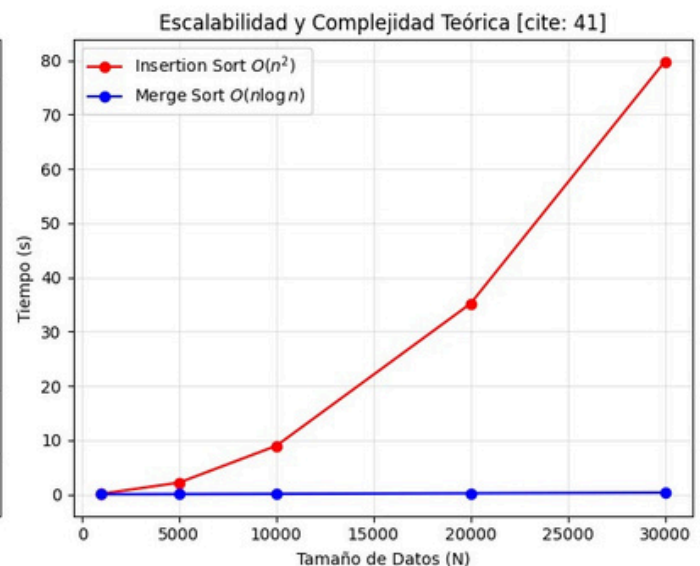
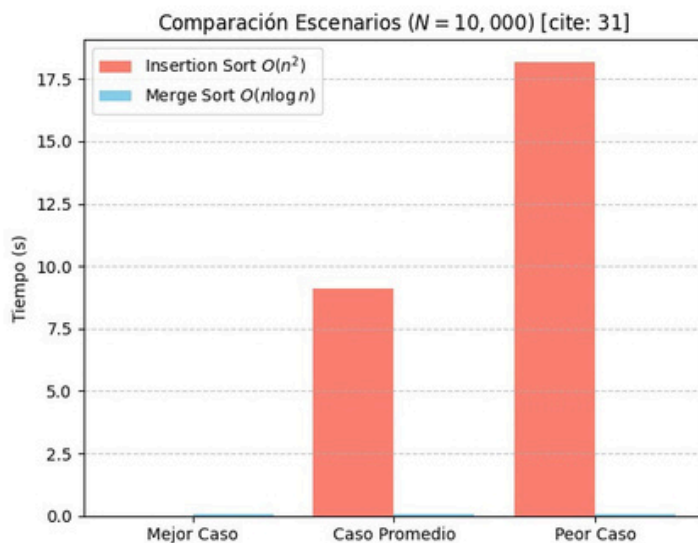
- Rendimiento estable en los tres escenarios.

08



RESULTADOS OBTENIDOS

6.4 Escalabilidad y Complejidad Teórica

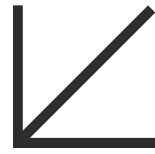


La gráfica comparativa muestra cómo el algoritmo $O(n^2)$ se degrada severamente en el peor caso, mientras que $O(n \log n)$ mantiene estabilidad.



09

PSEUDOCÓDIGOS



Insertion Sort — $O(n^2)$

```
INSERTION_SORT(A)
  para i ← 1 hasta longitud(A) - 1 hacer
    clave ← A[i]
    j ← i - 1
    mientras j ≥ 0 y A[j] > clave hacer
      A[j + 1] ← A[j]
      j ← j - 1
    fin mientras
    A[j + 1] ← clave
  fin para
```

Merge Sort — $O(n \log n)$

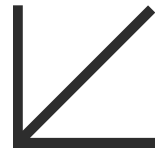
```
si longitud(A) > 1 entonces
  mitad ← longitud(A) / 2
  L ← primera mitad de A
  R ← segunda mitad de A
  MERGE_SORT(L)
  MERGE_SORT(R)
  MERGE(A, L, R)
fin si
```

Función merge

```
IMERGE(A, L, R)
  i ← 0, j ← 0, k ← 0
  mientras i < longitud(L) y j < longitud(R) hacer
    si L[i] ≤ R[j] entonces
      A[k] ← L[i]
      i ← i + 1
    sino
      A[k] ← R[j]
      j ← j + 1
    fin si
    k ← k + 1
  fin mientras
  copiar elementos restantes de L o R en A
```

10

PSEUDOCÓDIGOS



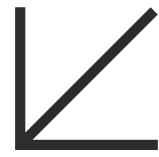
Búsqueda Binaria — $O(\log n)$

```
BUSQUEDA_BINARIA(A, x)
  bajo ← 0
  alto ← longitud(A) - 1
  mientras bajo ≤ alto hacer
    medio ← (bajo + alto) / 2
    si A[medio] = x entonces
      retornar medio
    sino si A[medio] < x entonces
      bajo ← medio + 1
    sino
      alto ← medio - 1
  fin si
fin mientras
retornar -1
```

Búsqueda Secuencial — $O(n)$

```
BUSQUEDA_SECUENCIAL(A, x)
  para i ← 0 hasta longitud(A) - 1 hacer
    si A[i] = x entonces
      retornar i
  fin si
fin para
retornar -1
```

11



PSEUDOCÓDIGOS

Pila (Stack) — $O(1)$

PUSH(Pila, x)

agregar x al final de la pila

POP(Pila)

si pila no está vacía entonces

eliminar y retornar último elemento

sino

retornar NULL

Cola (Queue) — $O(1)$

Enqueue

ENQUEUE(Cola, x)

agregar x al final de la cola

Dequeue

DEQUEUE(Cola)

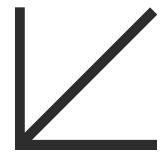
si cola no está vacía entonces

eliminar y retornar primer elemento

sino

retornar NULL

12

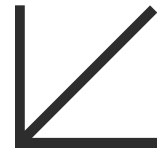


CONCLUSIONES, Y VALIDACIÓN TEÓRICA

A partir de las mediciones realizadas mediante temporización de alta precisión, se comprobó que los tiempos de ejecución observados siguen el comportamiento esperado según el análisis estudiado en la clase de Estructura de Datos.

En el caso de los algoritmos de ordenamiento, los experimentos confirmaron que Insertion Sort, con complejidad $O(n^2)$, presenta un crecimiento acelerado en el tiempo de ejecución conforme aumenta el tamaño de los datos, especialmente en el peor caso. Este comportamiento evidencia que, aunque puede ser eficiente para conjuntos pequeños o casi ordenados, no resulta viable para volúmenes grandes de información. En contraste, Merge Sort, con complejidad $O(n \log n)$, mostró un rendimiento estable en los escenarios de mejor, promedio y peor caso, validando su eficiencia y escalabilidad, así como la importancia de elegir algoritmos con complejidades más favorables para aplicaciones reales.

13

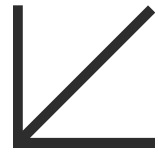


CONCLUSIONES, Y VALIDACIÓN TEÓRICA

Respecto a los algoritmos de búsqueda, los tiempos registrados demostraron claramente la ventaja de la búsqueda binaria frente a la búsqueda secuencial. Mientras que la búsqueda secuencial presentó un comportamiento lineal, la búsqueda binaria mantuvo tiempos significativamente menores gracias a su complejidad logarítmica. Esta diferencia confirma teóricamente que el preprocesamiento de los datos (ordenamiento previo) puede generar mejoras sustanciales en el rendimiento general de un sistema.

En cuanto a las estructuras lineales, las operaciones fundamentales de Pilas (Push y Pop) y Colas (Enqueue y Dequeue) mostraron tiempos de ejecución prácticamente constantes, independientemente del número de elementos almacenados. Estos resultados validan empíricamente la complejidad algorítmica lineal de dichas operaciones, demostrando que el acceso y modificación en estas estructuras no depende del tamaño de la colección, sino únicamente de la operación ejecutada. Esto resalta su eficiencia y su uso frecuente en aplicaciones que requieren rapidez y control del flujo de datos.

14



CONCLUSIONES, Y VALIDACIÓN TEÓRICA

En conclusión, este proyecto demuestra que la elección del algoritmo y de la estructura de datos adecuada impacta directamente en el rendimiento, escalabilidad y eficiencia de una aplicación. La validación experimental realizada refuerza la importancia del análisis de complejidad como herramienta esencial en el diseño de soluciones computacionales eficientes, permitiendo tomar decisiones fundamentadas que optimizan el uso de recursos y el tiempo de ejecución en sistemas reales.